

Informe de Cumplimiento de la Rubrica

Proyecto

Ospedale

Objetivo del documento

Este informe explica, de manera explicita, como se cumple cada inciso del parcial en el proyecto actual. Tambien describe como se aplican MVC, SOLID, algunos patrones de diseno y en que partes del codigo se evidencia cada decision.

1. Estructura general del proyecto

La arquitectura fue reorganizada alrededor de MVC con capas de apoyo:

- src/packagee/ospedale/model: entidades del dominio.
- src/packagee/ospedale/view: interfaces Swing.
- src/packagee/ospedale/controller: controladores delgados que reciben datos de la vista y delegan.
- src/packagee/ospedale/service: reglas de negocio y casos de uso.
- src/packagee/ospedale/repository: acceso al almacenamiento simulado.
- src/packagee/ospedale/validator: validaciones de entrada.
- src/packagee/ospedale/factory: construccion de objetos.
- src/packagee/ospedale/observer: observador para refresco automatico.
- src/packagee/ospedale/model/storage: almacenamiento central y serializacion.

2. Cumplimiento de la rubrica en vistas

2.1 La vista no se ejecuta a si misma

El proyecto arranca desde src/main/Main.java, clase Main, metodo main.

La vista de login no arranca por su cuenta. Se crea desde Main y se muestra con `new Login().setVisible(true)`.

2.2 Navegacion entre vistas segun el rol

La vista Login realiza el ingreso y navega segun el rol retornado por el controlador:

- src/packagee/ospedale/view/Login.java, metodo `enter_login1ActionPerformed`
- src/packagee/ospedale/controller/AuthController.java, metodo `login`
- src/packagee/ospedale/service/AuthService.java, metodo `login`

Si el rol es:

- admin: abre Admin_View
- doctor: abre Doctor_View
- patient: abre Patient_View

El logout lleva de vuelta a Login en:

- src/packagee/ospedale/view/Admin_View.java
- src/packagee/ospedale/view/Doctor_View.java
- src/packagee/ospedale/view/Patient_View.java

2.3 El administrador puede entrar a vista doctor y paciente

Esto se cumple desde Admin_View:

- doctor_view_buttonActionPerformed
- patient_view_buttonActionPerformed

Archivos:

- src/packagee/ospedale/view/Admin_View.java

Se envia el id correspondiente al constructor de la vista y ademas un booleano fromAdmin.

2.4 El boton back solo esta activo para admin

Esto se controla en constructores:

- src/packagee/ospedale/view/Doctor_View.java
- src/packagee/ospedale/view/Patient_View.java

En ambos casos se usa:

- back_button.setVisible(fromAdmin)
- backButton.setVisible(fromAdmin)

2.5 La informacion del usuario se carga automaticamente

Paciente:

- src/packagee/ospedale/view/Patient_View.java, metodo loadPatientInfo

Doctor:

- src/packagee/ospedale/view/Doctor_View.java, metodo loadDoctorInfo

Estas cargas usan respuestas serializadas, no objetos del modelo directamente.

2.6 ComboBox cargados automaticamente

Ejemplos:

- Admin_View.loadDoctorsComboBox
- Admin_View.loadPatientsComboBox
- Patient_View.loadDoctorsComboBox
- Patient_View.loadAppointmentsComboBox
- Patient_View.loadRoomTypesComboBox
- Doctor_View.loadAppointmentsComboBoxes
- Doctor_View.loadPatientsComboBox
- Doctor_View.loadHospitalizationsComboBox

2.7 La vista invoca controladores y espera respuesta

Esto se cumple en todas las vistas. Ejemplos:

- Login.save_patient_registerActionPerformed llama PatientController.registerPatient
- Login.enter_login1ActionPerformed llama AuthController.login
- Patient_View.createRequestMedicalAppointmentActionPerformed llama MedicalServiceController.requestAppointment
- Doctor_View.accept_medical_appointment_buttonActionPerformed llama MedicalServiceController.acceptAppointment

2.8 La vista notifica al usuario el resultado

Las vistas muestran mensajes usando JOptionPane.

Ejemplos:

- Login.showResponse
- mensajes de exito y error en Admin_View, Patient_View y Doctor_View

2.9 Si la respuesta es exitosa, se limpian componentes

Ejemplos:

- Login.clearPatientRegisterFields
- Login.clearLoginFields
- limpieza despues de crear cita en Patient_View
- limpieza despues de hospitalizacion en Patient_View
- limpieza despues de aceptar, completar o reagendar en Doctor_View

2.10 Restriccion visual

No se replanteo el layout visual base; se reutilizaron los formularios Swing existentes. La refactorizacion se concentro en logica, navegacion, carga de datos y conexion correcta con controladores.

3. Cumplimiento de la rubrica en controladores

3.1 Uso del sistema de respuestas y estados

Se mantiene el esquema de Response y Status:

- `src/packagee/ospedale/controller/utils/Response.java`
- `src/packagee/ospedale/controller/utils/Status.java`

Todos los controladores retornan Response.

3.2 Controladores implementados

Login:

- `AuthController.login`

Pacientes:

- `PatientController.registerPatient`
- `PatientController.updatePatient`
- `PatientController.getPatientInfo`
- `PatientController.getAllPatients`

Doctores:

- `DoctorController.registerDoctor`
- `DoctorController.updateDoctor`
- `DoctorController.getDoctorInfo`
- `DoctorController.getAllDoctors`

Servicios medicos:

- `MedicalServiceController.requestAppointment`
- `acceptAppointment`
- `completeAppointment`
- `cancelAppointment`
- `rescheduleAppointment`

- prescribeMedication
- requestHospitalization
- acceptHospitalization
- cancelHospitalization
- hospitalizeFromAppointment
- getPatientAppointments
- getDoctorAppointments
- getPatientHospitalizations
- getRequestedHospitalizations

3.3 Reglas funcionales exigidas

Estas reglas viven principalmente en servicios:

- AuthService
- PatientService
- DoctorService
- AppointmentService
- HospitalizationService

Unicidad y validacion de usuarios

- ids de 12 digitos y mayores que 0:
- validator/UserValidator.java
- username unico:
- PatientService
- DoctorService
- contrasena y confirmacion:
- UserValidator.validatePassword

Validaciones de paciente

- email:
- PatientValidator.validateEmail
- telefono:
- PatientValidator.validatePhone
- fecha de nacimiento:
- PatientValidator.validateBirthdate

Validaciones de doctor

- licencia:
- DoctorValidator.validateLicence
- oficina:
- DoctorValidator.validateOffice

Citas

- solicitud de cita por doctor o por especialidad:
- AppointmentService.requestAppointment
- validacion de fecha:
- AppointmentService.validateDate
- validacion de hora y cuartos de hora:
- AppointmentService.validateTime
- disponibilidad del doctor:
- Storage.isDoctorAvailable
- Storage.isDoctorAvailableExcluding
- ids automaticos A-{id_paciente}-NNNN:
- Storage.generateAppointmentId
- estado inicial REQUESTED:
- constructor de Appointment
- aceptar cita:
- AppointmentService.acceptAppointment
- completar cita:
- AppointmentService.completeAppointment
- cancelar cita:
- AppointmentService.cancelAppointment
- reagendar cita:
- AppointmentService.rescheduleAppointment
- prescribir medicamentos:
- AppointmentService.prescribeMedication

Hospitalizaciones

- ids automaticos H-{id_paciente}-NNNN:
- Storage.generateHospitalizationId
- solicitud:
- HospitalizationService.requestHospitalization
- estado inicial REQUESTED si la solicita un paciente:
- constructor de Hospitalization

- aprobar hospitalizacion:
- HospitalizationService.acceptHospitalization
- denegar hospitalizacion:
- HospitalizationService.cancelHospitalization
- hospitalizar desde cita y completar cita:
- AppointmentService.hospitalizeFromAppointment

Tablas y visualizacion

- citas de paciente ordenadas descendentemente:
- Storage.getAppointmentsByPatientSorted
- citas de doctor ordenadas descendentemente:
- Storage.getAppointmentsByDoctorSorted
- filtro de pendientes para doctor:
- Storage.getAppointmentsByDoctorSorted

3.4 No enviar modelos directos a la vista

Esto se cumple mediante serializacion:

- Storage.serializeUser
- Storage.serializePatient
- Storage.serializeDoctor
- Storage.serializeAppointment
- Storage.serializeHospitalization

Las vistas leen HashMap<String, Object> retornados por Response.

4. Cumplimiento de la rubrica en modelos

4.1 Modelos de dominio

Principales entidades:

- User
- Administrator
- Patient
- Doctor
- Appointment
- Hospitalization
- Prescription

Enumeraciones:

- AppointmentStatus
- HospitalizationStatus
- Specialty
- RoomType

4.2 Almacenamiento simulado

Se centraliza en:

- src/packagee/ospedale/model/storage/Storage.java

Este componente:

- carga usuarios desde JSON
- conserva listas en memoria
- genera ids automáticos
- serializa datos para la vista
- coordina disponibilidad y ordenamiento
- emite eventos para observadores

4.3 Carga desde JSON y construccion de objetos

La carga se hace en:

- Storage.loadUsersFromJson

El archivo fuente es:

- json/users.json

5. MVC aplicado en el proyecto

5.1 Modelo

Representado por:

- model
- repository
- validator
- factory
- model/storage

5.2 Vista

Representada por:

- Login
- Admin_View
- Patient_View
- Doctor_View

La vista:

- captura datos
- invoca controladores
- muestra respuestas
- actualiza tablas y combos

5.3 Controlador

Representado por:

- AuthController
- PatientController
- DoctorController
- MedicalServiceController

Su responsabilidad es recibir peticiones de la vista y delegarlas a servicios.

6. SOLID aplicado en el proyecto

6.1 Single Responsibility Principle

Cada capa tiene una responsabilidad clara:

- controladores: orquestacion de entrada/salida
- servicios: reglas de negocio
- validadores: reglas de formato y consistencia
- repositorios: acceso a Storage
- storage: persistencia simulada y serializacion
- vistas: interfaz y eventos graficos

Ejemplos:

- PatientService maneja casos de uso de paciente

- DoctorValidator solo valida doctores
- AppointmentRepositoryImpl solo accede a almacenamiento para citas

6.2 Open/Closed Principle

La separacion por servicios, validadores y repositorios permite extender comportamientos sin alterar toda la UI.

Ejemplo:

- se agrego HospitalizationService sin convertir a la vista o al controlador en componentes gigantescos

6.3 Liskov Substitution Principle

Administrator, Patient y Doctor heredan de User y pueden ser tratados como usuario base cuando corresponde.

Ejemplo:

- Storage.serializeUser
- almacenamiento general de usuarios en ArrayList<User>

6.4 Interface Segregation Principle

Los repositorios usan interfaces pequenas y especificas:

- PatientRepository
- AppointmentRepository

No se obliga a una clase a implementar metodos que no necesita.

6.5 Dependency Inversion Principle

Los servicios dependen de abstracciones de repositorio y no directamente de la implementacion concreta en todos los casos clave.

Ejemplos:

- PatientService depende de PatientRepository
- AppointmentService depende de AppointmentRepository

7. Patrones de diseño usados

7.1 MVC

Patron principal del proyecto.

7.2 Repository

Separacion entre casos de uso y acceso a almacenamiento:

- PatientRepository
- AppointmentRepository

7.3 Factory

Creacion centralizada de pacientes:

- PatientFactory.createPatient

7.4 Singleton

El almacenamiento usa instancia unica:

- Storage.getInstance

7.5 Observer

Se implemento para el bono de actualizacion automatica de tablas y combos:

- StorageObserver
- StorageEventType
- Storage.addObserver
- Storage.removeObserver
- Storage.publishEvent

Registro del observador en vistas:

- Admin_View.registerObserver
- Patient_View.registerObserver
- Doctor_View.registerObserver

8. Bonificaciones

8.1 SOLID en controladores

Se cumple en mayor medida que antes porque los controladores quedaron como capa delegadora:

- AuthController delega a AuthService
- DoctorController delega a DoctorService
- MedicalServiceController delega a AppointmentService y HospitalizationService
- PatientController delega a PatientService

Esto evita que la vista o el controlador concentren reglas de negocio pesadas.

8.2 Patron observador para actualizar tablas automaticamente

Se implemento el observador en el almacenamiento y las vistas se suscriben a eventos.

Eventos:

- USERS_CHANGED
- APPOINTMENTS_CHANGED
- HOSPITALIZATIONS_CHANGED

Cuando una operacion cambia el estado del sistema, los servicios publican el evento correspondiente.

Ejemplos:

- PatientService publica USERS_CHANGED
- DoctorService publica USERS_CHANGED
- AppointmentService publica APPOINTMENTS_CHANGED
- HospitalizationService publica HOSPITALIZATIONS_CHANGED

Las vistas escuchan y recargan:

- combos
- informacion de usuario
- tablas
- listas de hospitalizacion

9. Flujo practico de llenado de tablas

Las tablas no vienen llenas con citas u hospitalizaciones de ejemplo. Se llenan al usar el sistema.

Flujo recomendado:

1. Entrar como paciente.
2. Solicitar una cita u hospitalizacion.
3. Entrar como doctor.
4. Aceptar, reagendar, completar o prescribir.
5. Ver como las tablas se actualizan.

El admin tambien puede navegar a vista doctor o paciente para probar rapidamente.

10. Conclusion

El proyecto actual cumple la idea central del parcial mediante una arquitectura MVC apoyada en servicios, validadores, repositorios, almacenamiento simulado, serializacion y observadores. Ademas, se cubren los casos funcionales exigidos por la rubrica y se implementan los bonos de SOLID en controladores y actualizacion automatica por patron observador.

11. Archivos mas importantes para sustentar

- src/main/Main.java
- src/packagee/ospedale/controller/AuthController.java
- src/packagee/ospedale/controller/PatientController.java
- src/packagee/ospedale/controller/DoctorController.java
- src/packagee/ospedale/controller/MedicalServiceController.java
- src/packagee/ospedale/service/AuthService.java
- src/packagee/ospedale/service/PatientService.java
- src/packagee/ospedale/service/DoctorService.java
- src/packagee/ospedale/service/AppointmentService.java
- src/packagee/ospedale/service/HospitalizationService.java
- src/packagee/ospedale/model/storage/Storage.java
- src/packagee/ospedale/view/Login.java
- src/packagee/ospedale/view/Admin_View.java
- src/packagee/ospedale/view/Patient_View.java
- src/packagee/ospedale/view/Doctor_View.java